QUYẾT ĐỊNH THIẾT KẾ VÀ GIẢ ĐỊNH

Hợp nhất `spec_1` và `spec_2` thành một thiết kế baseline duy nhất

Mục lục

1 Phạm vi và cách dùng tài liệu 2

2 Kiểm kê nguồn 2

3 Giải quyết mâu thuẫn giữa hai nguồn 4

4 Quyết định do người soạn baseline tự chọn 6

5 Sơ đồ, waveform và công cụ 9

6 Điểm còn phải xác nhận 10

7 Giới hạn của bộ tài liệu 10

1 Phạm vi và cách dùng tài liệu

Tài liệu này ghi lại *vì sao* bộ spec baseline nói như nó đang nói. Nó gồm ba phần: bảng kiểm kê nguồn (mục 2), bảng giải quyết mâu thuẫn giữa `spec_1` và `spec_2` (mục 3), và bảng các quyết định do người soạn baseline tự chọn khi nguồn không nêu (mục 4). Mỗi quyết định có một mã `D-xx`; các tài liệu system và module dẫn chiếu đúng mã đó trong hộp *Quyết định baseline*.

Quy tắc ưu tiên đã dùng

- **Nội dung kỹ thuật**: `spec_1` và `spec_2` ngang quyền. Không nguồn nào được coi là đúng hơn chỉ vì là nguồn nào
- **Bố cục, cách chia tài liệu, phong cách trình bày, phong cách bảng / sơ đồ / waveform / nhãn**: theo `spec_1`
- Khi hai nguồn mâu thuẫn: trước hết xét lựa chọn nào phù hợp hơn với kiến trúc đã chốt; nếu cả hai đều phù hợp thì chọn theo tính nhất quán toàn hệ thống, khả năng triển khai, độ rõ ràng của giao diện và mức độ hỗ trợ từ tài liệu nguồn
- Khi nguồn không nêu: chọn một giá trị hoặc quy tắc baseline phù hợp với kiến trúc đã chốt, ghi rõ là quyết định do người soạn chọn, kèm lý do và tác động
- Không giữ hai phương án ngang hàng ở bất kỳ chỗ nào trong bộ tài liệu

Kiến trúc đã chốt (tiền đề, không phải quyết định)

- Đúng sáu khối chức năng, xác định từ `spec_1/modules/`
- DMA theo chuỗi descriptor; Local SRAM ping-pong; systolic array 8×8 PE
- `clk` mục tiêu 100 MHz; ma trận vào INT8; tích lũy và C ra INT32
- Giữ nguyên port của accelerator top trong `spec_1` ở mức tối đa

2 Kiểm kê nguồn

Bảng 2.1: Kiểm kê nguồn `spec_1`

Đường dẫn	Loại	Chủ đề	Trạng thái đọc
`spec_1/system/latex/main.tex`	LaTeX	Thứ tự mục của tài liệu hệ thống	Đã đọc
`spec_1/system/latex/preamble.tex`	LaTeX	Font, màu, bảng, hộp, TikZ; nguồn của `common/spec_style.tex`	Đã đọc
`spec_1/system/latex/frontmatter/titlepage.tex`	LaTeX	Trang bìa	Đã đọc
`spec_1/system/latex/sections/s1_tong_quan.tex`	LaTeX	Phạm vi, quy ước, module, tham số hệ thống	Đã đọc
`spec_1/system/latex/sections/s2_chuc_nang.tex`	LaTeX	Phép toán, kiểu dữ liệu, bố trí DRAM, mô hình tham chiếu	Đã đọc
`spec_1/system/latex/sections/s3_lap_trinh.tex`	LaTeX	Register map, bit field, mã lỗi, ngữ nghĩa tác vụ, quy tắc firmware	Đã đọc
`spec_1/system/latex/sections/s4_luong_van_hanh.tex`	LaTeX	Các pha, quyền truy cập scratchpad, xử lý lỗi, reset	Đã đọc
`spec_1/system/latex/sections/s5_cac_khoi.tex`	LaTeX	Bảng kết nối giữa các module	Đã đọc

Bảng 2.1: Kiểm kê nguồn `spec_1` (tiếp theo)

Đường dẫn	Loại	Chủ đề	Trạng thái đọc
`spec_1/system/latex/sections/s6_port_top.tex`	LaTeX	44 port top đã khóa, tham số top	Đã đọc
`spec_1/system/latex/sections/s7_bo_sung.tex`	LaTeX	Bố cục descriptor 16 word, chuỗi, ping-pong	Đã đọc
`spec_1/modules/register_file/latex/main.tex`	LaTeX	Port, thanh ghi, timing, waveform `fig:6-1`	Đã đọc
`spec_1/modules/main_fsm/latex/main.tex`	LaTeX	Port, `job_t`, bảng trạng thái ba engine, timing	Đã đọc
`spec_1/modules/dma_engine/latex/main.tex`	LaTeX	Port, `gemm_dma_agen`, kênh đọc, kênh ghi	Đã đọc
`spec_1/modules/local_sram/latex/main.tex`	LaTeX	Port, bank và sub-bank, ánh xạ địa chỉ, timing	Đã đọc
`spec_1/modules/systolic_array/latex/main.tex`	LaTeX	Port, PE, skew, tile sequencer, drain, timing	Đã đọc
`spec_1/modules/vpu/latex/main.tex`	LaTeX	Port, đường ống 4 tầng, requantize, timing	Đã đọc
`spec_1/modules/systolic_array/latex/preamble.tex`, `.../vpu/latex/preamble.tex`	LaTeX	Biến thể preamble cho module	Đã đọc
`spec_1/*/latex/latexmkrc` (7 file)	Cấu hình	XeLaTeX, `pdf_mode=5`	Đã đọc
`spec_1/system/gemm_system_spec_v1.0.pdf` và 6 PDF module	PDF	Bản kết xuất của các `main.tex` trên	**Không đọc** — là bản in của nguồn LaTeX đã đọc đầy đủ; không chứa thông tin thêm
`spec_1/common/timer_style.tex`	LaTeX	File style dùng chung mà cả 7 `main.tex` đều `\textbackslashinput`	**Không tồn tại** trên đĩa; tìm toàn bộ cây thư mục không thấy

Bảng 2.2: Kiểm kê nguồn `spec_2`

Đường dẫn	Loại	Chủ đề	Trạng thái đọc
`spec_2/00_system_overview.html`	HTML	Kiến trúc, tham số PAR-001…015, register map, luồng thực thi, yêu cầu REQ-*, quyết định DEC-001…013	Đã đọc
`spec_2/MOD-001_gemm_accel_top.html`	HTML	Port top, net nội bộ, bộ đồng bộ reset	Đã đọc
`spec_2/MOD-002_gemm_register_file.html`	HTML	Port, chấp nhận START, cập nhật STATUS, timing	Đã đọc
`spec_2/MOD-003_gemm_main_fsm.html`	HTML	Port, scoreboard bank, FSM điều khiển và ba engine	Đã đọc
`spec_2/MOD-004_gemm_dma_engine.html`	HTML	Port, `gemm_dma_agen`, FSM đọc/ghi, cắt burst, WFIFO	Đã đọc
`spec_2/MOD-005_gemm_local_sram.html`	HTML	Port, sub-bank, ánh xạ, trọng tài, assertion	Đã đọc
`spec_2/MOD-006_gemm_systolic_array.html`	HTML	Port, PE, skew, tile_seq, drain, lịch timing	Đã đọc
`spec_2/MOD-007_gemm_vpu.html`	HTML	Port, đường ống, requantize, assertion AS-01…AS-07	Đã đọc

Bảng 2.2: Kiểm kê nguồn `spec_2` (tiếp theo)

Đường dẫn	Loại	Chủ đề	Trạng thái đọc
`spec_2/*.pdf` (8 file)	PDF	Bản kết xuất của 8 file HTML trên	**Không đọc** — nội dung trùng HTML đã đọc đầy đủ
`spec_2/../figures/figNN.svg` (19 hình)	SVG	19 hình được `<imgsrc>` tham chiếu từ các file HTML	**Không đọc được** — thư mục `figures/` không tồn tại; các PDF cũng không chứa nội dung hình. Xem mục 5

TRẠNG THÁI CỦA SPEC_2
Các file `spec_2` tự ghi tiêu đề *DRAFT — NOT APPROVED* và có mâu thuẫn nội bộ: các yêu cầu REQ-CFG-001, REQ-CFG-002, REQ-MEM-001, REQ-REG-001, REQ-REG-002 vẫn mang nhãn UNRESOLVED (ISS-001…ISS-005) dù các quyết định DEC-001…DEC-005 trong cùng tài liệu đã chốt chính các điểm đó, trong khi phần đầu tài liệu ghi "Open issues (0)". Điều này **không** làm giảm quyền của nội dung kỹ thuật `spec_2`; nhưng ở mỗi chỗ `spec_2` tự mâu thuẫn, baseline lấy phần đã được DEC-xxx chốt.

3 Giải quyết mâu thuẫn giữa hai nguồn

Bảng 3.1: Mâu thuẫn và lựa chọn baseline

Mã	Chủ đề	Thông tin từ nguồn / mâu thuẫn	Lựa chọn baseline	Lý do
D-01	Mô hình lập trình	`spec_1`: chỉ chuỗi descriptor, 0x08…0x24 Reserved, `CTRL[2]` = START. `spec_2`: thêm chế độ single với 8 thanh ghi RW tại 0x08…0x24, `CTRL[0]` = START, `CTRL[2]` = START_CHAIN, `STATUS[3]` = MODE	Chỉ chuỗi descriptor. 0x08…0x24 Reserved. `CTRL[2]` = START, `CTRL[1]` = SOFT_RESET, `CTRL[0]` và `STATUS[3]` dự trữ. RF→FSM chỉ còn `o_start_pulse`, `o_soft_req`, `o_cfg_desc_ptr`	Kiến trúc chốt là descriptor-based DMA. Chế độ register-programmed là nhánh thứ hai làm đúng cùng việc, nhân đôi đường cấu hình trong RF và FSM (thêm 8 bus cấu hình ứng với 8 thanh ghi RW, cùng tín hiệu `start_kind`), và buộc STATUS mang thêm bit chế độ
D-02	Miền `ERR_CODE`	`spec_1`: 1…4, 5…15 dự trữ. `spec_2`: thêm mã 5 "tính năng không có", tự ghi là không bao giờ phát sinh	0…4; 5…15 dự trữ, không bao giờ xuất hiện	Hệ quả trực tiếp của D-01: một chế độ duy nhất thì không có tình huống "tính năng không có"
D-03	`HW_CONFIG`	`spec_1`: {8'h11, 8'h00, 8'd32, P}, SPEC_VER 0x11. `spec_2`: 0x1003_2008, SPEC_VER 0x10, thêm `HAS_CHAIN` bit 16 và `HAS_PINGPONG` bit 17	Reset 0x1100_2008; [23:16] Reserved	Khi chuỗi và ping-pong là chế độ duy nhất, hai bit `HAS_*` là hằng 1 và không mang thông tin; SPEC_VER = 0x11 đã mã hóa đúng khả năng đó bằng một trường. Hai cách mô tả cùng một việc tạo hai nguồn sự thật cho firmware

Bảng 3.1: Mâu thuẫn và lựa chọn baseline (tiếp theo)

Mã	Chủ đề	Thông tin từ nguồn / mâu thuẫn	Lựa chọn baseline	Lý do
D-04	Kích thước array P	spec_1: tham số nhận 4 hoặc 8, mặc định 8. spec_2: P = 8 cố định	P = 8 cố định; giá trị khác bị chặn lúc elaboration. Q = 2, LP = 3, AB_VEC_WIDTH = 64, ROW_VEC_WIDTH = 256, 64 PE	Kiến trúc chốt là PE 8×8. P = 4 kéo theo hai bộ độ rộng bus, hai lịch skew và hai công thức timing trong bốn module
D-05	MUL_STAGES, LAT, L_VPU	spec_1: MUL_STAGES 1 hoặc 2; LAT = 3 + MUL_STAGES; L_VPU mặc định 0 *nhưng* phải bằng LAT — mâu thuẫn nội bộ. spec_2: MUL_STAGES = 1, LAT = 4	MUL_STAGES = 1, LAT = L_VPU = 4; gemm_pkg::VPU_LATENCY là nguồn duy nhất, kiểm lúc elaboration ở cả array và VPU	Ở 100 MHz một tầng đủ cho nhân 32×32 → 64 bit. LAT nhỏ nhất, đường ống VPU nông nhất, khớp mặc định spec_2. MUL_STAGES = 2 là cách sửa timing đã định trước, không phải phương án song song
D-06	Định nghĩa r_last	spec_1: systolic_array dùng r_last = số hàng − 1; vpu dùng r_last = số hàng — mâu thuẫn nội bộ. spec_2: cho số kiểm chứng 534 chu kỳ với M = N = K = 32	r_last = (số hàng hợp lệ của tile cuối) − 1, tức **chỉ số**. Công thức: $(NT-1)\cdot \max(K,P)+K+P+r_{last}+LAT+3$	Chỉ định nghĩa chỉ số cho ra đúng 534 chu kỳ: $15\cdot 32+32+8+7+4+3=534$. Cách diễn đạt của vpu trong spec_1 lệch một chu kỳ và đã được sửa
D-07	Sinh địa chỉ 2D của DMA	spec_1: "nhân 6×6 hoặc cộng dồn". spec_2: cộng dồn, không dùng bộ nhân 32 bit	Cộng dồn trong **đường địa chỉ**: seg_base ← base lúc start, rồi seg_base ← seg_base + stride sau mỗi hàng. Không có bộ nhân trong đường địa chỉ	Một bộ cộng 32 bit thay cho nhân 6×16 rồi cộng: tốt hơn cho mục tiêu 100 MHz và cho diện tích
D-08	Giới hạn credit kênh ghi	spec_1 dùng hằng 4 trong điều kiện ghi, trong khi WFIFO_DEPTH cũng bằng 4; không nói dùng cái nào	o_spm_rd_valid = (rd_left > 0) && (fifo_cnt + inflight < WFIFO_DEPTH)	Tránh hằng số lặp: đổi độ sâu FIFO không làm sai điều kiện
D-09	Số bank NB	spec_1 (dma, main_fsm) có câu điều kiện "khi chỉ có một bank…" trong khi NB là localparam bằng 2	NB = 2 luôn luôn; bỏ hoàn toàn mọi câu điều kiện theo NB = 1	Kiến trúc chốt là ping-pong. Đường NB = 1 là mã chết và là nhánh thiết kế thứ hai
D-10	Độ rộng i_matrix_dim	spec_1: 32 bit ở biên array nhưng 24 bit ở gemm_tile_seq; bit [31:24] không nêu. spec_2: comp_dim = {8'b0, K, N, M}	Port 32 bit ở biên module, nội dung {8'h00, K[7:0], N[7:0], M[7:0]}; [31:24] luôn 0, array bỏ qua; gemm_tile_seq chỉ dùng 24 bit thấp	Giữ port như spec_1 (không đổi giao diện), đồng thời nói rõ bit nào có nghĩa. Main FSM đã kiểm DIM[31:24] = 0 trước khi phát
D-11	DESC_PTR lệch căn	Hai nguồn trùng nhau	DESC_PTR[5:0] ≠ 0 → ERR_CODE = 1, không truy cập bus, không truy cập scratchpad	Không có mâu thuẫn; ghi lại để khóa hành vi
D-12	Trọng tài cổng SRAM	spec_2: en = dma_sel | cmp_sel, DMA thắng khi trùng, kèm assertion AS-02 rằng điều đó không xảy ra. spec_1 chỉ có assertion	DMA thắng khi trùng **và** giữ assertion AS-02	Hành vi RTL phải xác định, không để "không xác định"; assertion mới là cơ chế phát hiện lỗi lập lịch

Bảng 3.2: Nội dung chỉ có ở một nguồn và được giữ

Mã	Chủ đề	Xử lý
D-13	Chỉ có ở `spec_2`	Mục tiêu 100 MHz (DEC-013, PAR-015). Ghi chú ứng dụng chia ma trận lớn hơn 32 theo M, N, K (REQ-AP-001) → mục *Ma trận lớn hơn 32* của tài liệu hệ thống. Bảng kiểm kê bộ nhớ vật lý MEM-001…005. Ước lượng tài nguyên VPU (8 bộ nhân 32×32, 32 DSP). Danh sách assertion AS-01…AS-07 của VPU, AS-01/AS-02 của SRAM, AS-01 của array. Số liệu timing: COMPUTE 534 chu kỳ cho 32^3; DMA 1 beat/chu kỳ; `i_wr_start` tại t → `awvalid` và lần đọc SRAM đầu tại t+2, beat W đầu sớm nhất t+4
D-14	Chỉ có ở `spec_1`	Bố cục descriptor 16 word đầy đủ (`spec_2` xác nhận 16 word và 64 byte nhưng không nêu offset từng word và từng bit). Mô hình tham chiếu numpy. Hệ quả không tràn $\|C\| \leq 2^{19}$. Ghi chú tích hợp Chisel BlackBox, `GemmWrapper.scala`, cầu TLToAXI4 / AXI4ToTL, tín hiệu AXI phụ do wrapper cấp. Quy ước ba mức ”xong”

4 Quyết định do người soạn baseline tự chọn

Các quyết định trong mục này **không** có trong nguồn. Mỗi quyết định ghi rõ lý do và tác động. Mã D-15 … D-19 không dùng: dải D-0x/D-1x dành cho xung đột giữa hai nguồn và nội dung chỉ có ở một nguồn, dải D-2x/D-3x dành cho quyết định tự chọn; không có mục nào bị xóa hay gộp. D-34 và D-35 được thêm sau lượt review kiến trúc độc lập; D-36 do người dùng chốt.

Bảng 4.1: Quyết định baseline khi nguồn không nêu

Mã	Chủ đề	Quyết định	Lý do và tác động
D-20	Chu kỳ clock	`clk` 100 MHz, chu kỳ danh định 10,000 ns, duty 50 %. Mọi số đo thời gian trong bộ spec quy đổi theo 10 ns/chu kỳ	Nguồn không nêu chu kỳ; kiến trúc chốt đặt 100 MHz. Đường tổ hợp dài nhất đã biết: nhân 32×32 → 64 bit của VPU và MAC INT8 một chu kỳ của PE. Nếu tổng hợp không đạt, cách sửa đã định trước là `MUL_STAGES` = 2 (LAT = `L_VPU` = 5)
D-21	Giá trị khởi tạo bộ đồng bộ reset và quy tắc flip-flop	Hai flip-flop của bộ đồng bộ đều có reset bất đồng bộ về 0. Flip-flop trạng thái: `always_ff@(posedgeclkornegedgerst_n_sync)`. Flip-flop chỉ chứa dữ liệu (`acc`, `res` của PE, nội dung RAM, thanh ghi dữ liệu trong đường ống VPU, thanh ghi chọn bank của SRAM): `always_ff@(posedgeclk)`, không có giá trị reset. Không có latch	Nguồn nêu 2 flip-flop nhưng không nêu giá trị khởi tạo. Reset bất đồng bộ về 0 làm `rst_n_sync` xuống thấp ngay khi `rst_n` xuống, không cần clock chạy: accelerator vào trạng thái reset khi SoC chưa có clock ổn định. Không reset thanh ghi dữ liệu giúp giảm diện tích và không ảnh hưởng chức năng vì mọi đường dữ liệu đều có bit valid đi kèm
D-22	Nội dung scratchpad sau reset	Không xác định; không có mạch xóa	Xóa 3136 word cần một FSM và hàng nghìn chu kỳ mà không giúp gì: mọi vùng đều được ghi đầy trước khi đọc trong một job hợp lệ. Firmware không được đọc vùng đích trước DONE
D-23	Thứ tự descriptor và báo hoàn tất	Descriptor được FETCH đúng một lần theo đúng thứ tự chuỗi; STORE_C ra DRAM đúng theo thứ tự descriptor; `DESC_DONE_CNT` tăng 1 mỗi STORE_C xong OKAY; chuỗi kết thúc ở descriptor đầu tiên có `LAST` = 1. `o_done_pulse` và `o_err_pulse` không bao giờ cùng một chu kỳ	Nguồn nêu rải rác; gom thành một quy tắc để kiểm chứng được. Thứ tự ghi DRAM theo descriptor là điều kiện để bất biến ”giống hệt thực thi tuần tự” đúng cả khi các pha chạy chồng

Bảng 4.1: Quyết định baseline khi nguồn không nêu (tiếp theo)

Mã	Chủ đề	Quyết định	Lý do và tác động
D-24	Giới hạn địa chỉ và kiểm tra	Phần cứng không kiểm vùng nhớ có nằm trong DRAM, không kiểm chồng lấn. Chỉ kiểm các điều kiện cục bộ của descriptor	Kiểm chồng lấn cần biết toàn bộ bản đồ bộ nhớ SoC và so mọi cặp vùng của cả chuỗi: không khả thi trong accelerator. Tác động: vi phạm chồng lấn cho kết quả không xác định mà không báo lỗi; truy cập ngoài DRAM thành `ERR_CODE` = 2, 3 hoặc 4 qua phản hồi bus
D-25	Thông lượng baseline	Với M = N = K = 32, `POST_EN` = 0, stride = 0: 1552 beat AXI ≈ 15,52 µs cho một descriptor chạy đơn lẻ (đọc 528 beat, ghi 1024 beat; hai kênh chạy song song khi chồng pha) so với COMPUTE 534 chu kỳ ≈ 5,34 µs. Đỉnh array 64 MAC/chu kỳ = 6,4 GMAC/s	Hai nguồn đưa số liệu thành phần nhưng không tổng hợp. Kết luận thiết kế: tác vụ bị giới hạn bởi băng thông AXI 32 bit (khâu chậm nhất khi chạy chồng là kênh ghi, STORE_C ≈ 10,24 µs), không bởi array; ping-pong che được toàn bộ COMPUTE; cải thiện thông lượng phải đến từ độ rộng bus hoặc từ `POST_EN` = 1
D-26	Biên M, N, K không chia hết cho 8	Phần mềm đệm tới bội của **4** (N, K $\in$ {4,…,32}; M tùy ý 1…32). Phần cứng xử lý phần dư so với P = 8: $TM = \lceil M/8 \rceil$, $TN = \lceil N/8 \rceil$; hàng thừa bị chặn bằng `row_vld[r]` = (i0 + r < M); cột thừa bị chặn bằng `o_row_mask` (8 bit) → `i_row_mask` của VPU → `o_c_wr_mask` → `i_c_wr_mask` → byte/word enable của bank C. K không cần xử lý riêng	Đệm tới bội 4 là điều kiện đủ để mọi hàng DRAM trùng biên word, nhờ đó `m_axi_wstrb` luôn 4'b1111. Buộc phần mềm đệm tới bội 8 sẽ đòi thêm bộ nhớ mà phần cứng vốn đã có đủ cơ chế để chặn. Lưu ý hai cơ chế khác nhau: hàng thừa *không* cập nhật `acc` và không phát `trig`; cột thừa vẫn được tính nhưng không được ghi
D-27	Dấu, mở rộng dấu, overflow, saturation	Mọi phép cộng số nguyên **wrap**; chỉ bước tạo Y **saturate** về $[-128, 127]$. Tích INT8×INT8 là 16 bit có dấu, mở rộng dấu lên 32 bit. M_0 là INT32 có dấu (mọi giá trị hợp lệ), sh không dấu 0…31. Requantize hai bước: $t = prod \ggg (30 + sh)$ giữ đủ **34 bit**, $y = (t + 1) \ggg 1$ giữ đủ **33 bit**, rồi kẹp với $lo = \texttt{RELU_EN} \,?\, 0 : -128$	Nguồn nêu rải rác và không trùng cách diễn đạt; gom về một quy tắc duy nhất. Yêu cầu giữ 34/33 bit là bắt buộc: cắt t về 32 bit cho kết quả **sai** tại $acc = M_0 = -2^{31}$. Đây là lỗi hiện thực đã được nhận diện trong nguồn và baseline nâng thành yêu cầu kiểm chứng
D-28	Buffer, bank, sub-bank	NB = 2. Mỗi bank: A 256, B 256, C 1024, BIAS 32 word → 1568 word/bank, 3136 word = 12,25 KiB. Sub-bank: A 8×(32×32 bit), B 2×(128×32 bit), BIAS 8×(4×32 bit, register file), C 8×(128×32 bit có byte enable). Mỗi sub-bank là SRAM một cổng; độ trễ đọc đúng 1 chu kỳ; không có backpressure	Chốt lại các con số mà hai nguồn nêu nhất quán, thành một bảng duy nhất để module và system không lệch nhau. Một cổng là đủ vì mỗi bank chỉ có tối đa một bên truy cập mỗi chu kỳ (D-29)
D-29	Chọn và chuyển bank, đồng bộ producer/consumer	A, C đảo bank sau mỗi descriptor; B đảo sau mỗi LOAD_B (`REUSE_B` dùng lại bank trước); BIAS đảo sau mỗi descriptor có `POST_EN` = 1. Đồng bộ bằng scoreboard trong Main FSM: `rcnt[X][b]` 2 bit cho A, B, BIAS; `wcnt[C][b]` 1 bit. Trọng tài ba engine: **Ghi > Tính > Đọc**	Scoreboard là cơ chế duy nhất bảo đảm không bên nào đọc bank đang bị ghi hoặc ngược lại, và nó đủ tổng quát cho `REUSE_B` và `POST_EN` xen kẽ. Thứ tự ưu tiên Ghi > Tính > Đọc vì engine Ghi giải phóng bank C và engine Tính giải phóng bank A, B, BIAS: ưu tiên khâu giải phóng tài nguyên giữ đường ống chạy đầy

Bảng 4.1: Quyết định baseline khi nguồn không nêu (tiếp theo)

Mã	Chủ đề	Quyết định	Lý do và tác động
D-30	Nạp dữ liệu và lịch MAC của array	Output-stationary; A chảy sang đông, B chảy xuống nam, 1 hop/chu kỳ; skew cho lane A hàng r đúng r thanh ghi và lane B cột c đúng c thanh ghi (28 tầng mỗi phía, A 11 bit/tầng, B 8 bit/tầng); thứ tự tile tj trong, ti ngoài; slot `cnt` 0…I−1 với $I = \max(K, 8)$; MAC 1 chu kỳ, `i_first` nạp tích, `i_last` chốt `res`; xả bằng `trig[r]` one-hot, FIFO metadata sâu 2, MUX 8:1, một hàng 256 bit mỗi chu kỳ; `o_pe_done` = `o_final` trễ 5 chu kỳ	Gom các mảnh của hai nguồn thành một lịch đầy đủ và kiểm chứng được: đọc k của tile n ở chu kỳ S_n+k với $S_n = 1+n\cdot I$; hàng r của tile n xả ở $D_n(r) = S_n + K + P + 1 + r$. Hai công thức này cùng với D-06 cho đúng 534 chu kỳ
D-31	Cấu trúc, cách lấy và tiêu thụ descriptor	16 word = 64 byte, căn 64 byte, bố cục theo bảng của `spec_1`. Lấy bằng lệnh DMA FETCH: `o_rd_dst` = 3, `rows` = 1, `rwords` = 16, `stride` = 64, bank 0, một burst INCR 16 beat; trả về qua `o_desc_valid`, `o_desc_idx[3:0]`, `o_desc_data`; descriptor không vào scratchpad. Tiêu thụ: pha kiểm tra một chu kỳ dựng `job_t`; con trỏ kế tiếp từ `NEXT_PTR`; `LAST` = 1 kết thúc chuỗi	Hai nguồn trùng nhau về cơ chế; baseline chốt thành một trình tự duy nhất kèm giá trị từng trường của lệnh FETCH, để Main FSM và DMA không lệch nhau. Không cho descriptor vào scratchpad giữ bank A/B/BIAS chỉ chứa dữ liệu ma trận và không tốn một buffer thứ năm
D-32	Điều kiện descriptor hợp lệ	Gộp cả hai nguồn thành một danh sách đóng: miền M/N/K; `DIM[31:24]` = 0; mọi địa chỉ được dùng căn 4 byte; `NEXT_PTR` căn 64 byte khi `LAST` = 0; mỗi stride bằng 0 hoặc ($\geq$ độ dài hàng và bội của 4); mọi bit/word dự trữ bằng 0; `RELU_EN` kéo theo `POST_EN`; `REUSE_B` chỉ khi không phải descriptor đầu và K, N bằng của descriptor trước. `TAG` không bị kiểm	Danh sách phải đóng để `ERR_CODE` = 1 có nghĩa xác định và để kiểm chứng được. "Địa chỉ *được dùng*" là có chủ ý: `B_ADDR` không bị kiểm khi `REUSE_B` = 1 và `BIAS_ADDR` không bị kiểm khi `POST_EN` = 0. Ngoại lệ này chỉ áp dụng cho địa chỉ: cả ba stride luôn bị kiểm (kể cả `LDB` khi `REUSE_B` = 1), để bộ kiểm stride không phụ thuộc cờ
D-33	Trình tự soft reset	`CTRL[1]` → `o_soft_req` (ưu tiên hơn START trong cùng lệnh ghi; ở T_RUN ưu tiên hơn điều kiện DONE/ERROR cùng chu kỳ) → `T_SOFT`: không pha mới, không lệnh DMA mới, abort kênh đang bận, burst đã cam kết chạy hết, COMPUTE không được chờ → `T_CLR`: `o_soft_clr` một chu kỳ tới RF, DMA, SRAM, Array, VPU → `T_IDLE`. Trong cả chuỗi `o_soft_busy` = 1 và `STATUS` đọc 0x0000_0002. Sau đó `STATUS` = 0, không DONE, không ERROR; `DESC_PTR` và thanh ghi RW giữ nguyên	Nguồn nêu từng mảnh ở nhiều chỗ; gom thành một trình tự duy nhất có điều kiện kết thúc rõ ràng (SYS-09). FSM tự reset trong `T_CLR` nên không cần port `i_soft_clr`, giữ `o_soft_clr` là một net fan-out tới đúng năm khối

Bảng 4.1: Quyết định baseline khi nguồn không nêu (tiếp theo)

Mã	Chủ đề	Quyết định	Lý do và tác động
D-34	Dọn engine khi tác vụ kết thúc	Ở cạnh clock ctrl chuyển T_RUN → T_IDLE (DONE hoặc ERROR): engine Đọc → R_IDLE, engine Tính → C_IDLE, engine Ghi → S_IDLE, `slot_v` ← 0; job idx ≥ f còn sót bị bỏ. Assertion AS-15 của Main FSM: ở T_IDLE mọi engine ở IDLE và slot trống	Nguồn chỉ cho R_STOP thoát khi reset hoặc T_CLR và chỉ xóa bản ghi lỗi, bộ đếm, con trỏ bank khi START. Đọc theo chữ thì engine Đọc kẹt ở R_STOP nên tác vụ thứ hai treo (BUSY = 1 mãi), và sau ERROR job cũ trong slot hoặc C_HAND sẽ bị tính và ghi vào `C_ADDR` của tác vụ trước. Dọn ở cuối tác vụ (thay vì lúc START) để T_IDLE luôn là trạng thái sạch; an toàn vì điều kiện DONE/ERROR đã bảo đảm không kênh DMA bận và không COMPUTE đang chạy. Phát hiện bởi review kiến trúc (F-01, F-02)
D-35	Thành phần BIAS của AS-02 (SRAM)	Assertion vật lý giữ nguyên (không sub-bank nào bị DMA và phía tính chọn cùng chu kỳ). Dạng mức bank cho BIAS chỉ áp khi `i_bias_rd_en` = 1; với A, B, C áp mọi chu kỳ COMPUTE. Main FSM giữ quy tắc cấp bank BIAS như cũ (job `POST_EN` = 0 mang `bbias` = 0, không đảo bank)	Bản cũ ghi dạng mức bank là "tương đương" với dạng vật lý; sai khi `POST_EN` xen kẽ: job `POST_EN` = 0 đang tính vẫn mang `i_comp_bank_bias` = 0 trong khi LOAD_BIAS hợp lệ của job sau ghi bank 0, nên assertion báo lỗi giả. Phương án đổi cách gán `bbias` cho job `POST_EN` = 0 không được chọn vì vẫn còn trường hợp trùng bank khi có nhiều job `POST_EN` = 1 liền sau. Phát hiện bởi review kiến trúc (F-05)
D-36	`MAX_BURST_BEATS`	Chốt bằng 16 (burst tối đa 16 beat × 4 byte = 64 byte). Miền tham số 1 … 256 vẫn được giữ trong RTL để tái sử dụng, nhưng cấu hình baseline và mọi ví dụ, số liệu trong bộ spec dùng 16	Người dùng chốt ngày 2026-11-12. 16 beat khớp đúng một descriptor 64 byte (FETCH là một burst INCR 16). Trước đó điểm này nằm trong danh sách cần xác nhận

5 Sơ đồ, waveform và công cụ

Nguồn hình ảnh

- `spec_1` gần như không có sơ đồ: cả sáu tài liệu module cộng lại chỉ có **một** hình, là waveform `fig:6-1` trong `register_file` (ghi START rồi đọc STATUS). Tài liệu hệ thống của `spec_1` chỉ có các sơ đồ bit-field của thanh ghi. Hành vi được mô tả bằng bảng, danh sách và khối code
- `spec_2` tham chiếu 19 hình `figures/figNN.svg` nhưng thư mục `figures/` không tồn tại và các PDF không chứa nội dung hình, nên **không đọc được hình nào của `spec_2`**. Baseline không suy đoán nội dung các hình đó

Phong cách đã dùng

- **Waveform**: theo đúng phong cách `fig:6-1` của `spec_1` — đỏ `brand` cho tín hiệu 1 bit, ô xanh `data` cho bus nhiều bit, nhãn tín hiệu dạng `\textbackslashttfamily` căn phải, lưới chu kỳ đánh số từ 0. Bộ macro `\textbackslashsname`, `\textbackslashhi`, `\textbackslashlo`, `\textbackslashbus` của `spec_1` được đưa vào môi trường `wave` trong `common/spec_style.tex`, thêm `\textbackslashpulse` và `\textbackslashnote`, để mọi tài liệu dùng cùng một bộ
- **Sơ đồ khối và kiến trúc**: `spec_1` không có phần tương ứng. Baseline dùng skill `bw-diagram` (có sẵn trong môi trường) để tạo hai sơ đồ kiến trúc: `diagram/gemm_accel_top_dataflow.svg` (luồng dữ liệu) và `diagram/`

`gemm_accel_top_control.svg` (luồng điều khiển) — SVG đen trắng theo bộ ký hiệu cố định của skill, đã qua lint của skill với kết quả OK. Bản đầu tiên là một sơ đồ duy nhất rộng 2320 px, chữ in ra chỉ khoảng 2,7 pt nên được tách làm hai; mỗi canvas rộng không quá 1400 px để chữ 13 px in trên trang ngang A4 đạt khoảng 6,5–7 pt. Mỗi sơ đồ được chèn bằng `\textbackslashdiagfigwide` trên một trang nằm ngang, cần gói `svg` và Inkscape lúc biên dịch

- **Bit-field thanh ghi**: tái tạo đúng mẫu TikZ hai nửa (31…16 và 15…0) của `spec_1`
- Mọi tên tín hiệu, giao diện, thứ tự hoạt động và số chu kỳ trong sơ đồ và waveform khớp với phần mô tả tương ứng

File style dùng chung

- Cả bảy `main.tex` của `spec_1` đều `\textbackslashinput` file `spec_1/common/timer_style.tex`, nhưng file đó **không tồn tại**; vì vậy nguồn `spec_1` không biên dịch được như hiện trạng
- Baseline tái lập file style dùng chung thành `common/spec_style.tex` (trong thư mục baseline), dựng từ file `preamble.tex` ở `spec_1/system/latex/` (giữ nguyên font, bộ màu, `spectable`, `notebox`, `keypoint`, `qcard`, header và footer), tham số hóa phần header/footer qua `\textbackslashDocHeader`, `\textbackslashDocVersion`, `\textbackslashDocFooter`, `\textbackslashDocPdfTitle`, và bổ sung môi trường `wave` cùng lệnh `\textbackslashdiagfig`
- `\textbackslasheqbox` chỉ có trong preamble riêng của `systolic_array` và `vpu` trong `spec_1`; baseline không dùng lệnh này, thay bằng `notebox` hoặc công thức hiển thị thường

6 Điểm còn phải xác nhận

Danh sách này cố ý rất ngắn: mọi điểm còn lại đều đã được chốt thành một lựa chọn baseline. Chỉ các điểm dưới đây cần một thông tin từ bên ngoài bộ tài liệu.

Bảng 6.1: Điểm cần xác nhận

Chủ đề	Cần gì
Kết quả tổng hợp ở 100 MHz	Baseline chọn `MUL_STAGES` = 1 (D-05, D-20) dựa trên lập luận về đường tổ hợp, **không** dựa trên báo cáo tổng hợp. Cần một lần tổng hợp trên công nghệ đích để xác nhận; nếu không đạt, đặt `MUL_STAGES` = 2 và cập nhật LAT = `L_VPU` = 5 cùng các bảng timing phụ thuộc
Nội dung 19 hình của `spec_2`	Thư mục `figures/` không có trong workspace. Nếu các hình đó tồn tại ở nơi khác, cần đối chiếu để bảo đảm không có thông tin thiết kế nào chỉ nằm trong hình mà chưa có trong văn bản

7 Giới hạn của bộ tài liệu

PHẠM VI

Bộ spec baseline này là **tài liệu thiết kế**. Các mục ”kiểm chứng” trong tài liệu hệ thống và trong từng tài liệu module là *yêu cầu* kiểm chứng (assertion cần viết, điểm cần kiểm), không phải báo cáo kết quả. Trong quá trình soạn bộ tài liệu này:

- **Không** có RTL nào được viết, đọc hay đối chiếu
- **Không** có mô phỏng nào được chạy
- **Không** có tổng hợp nào được chạy; các con số 100 MHz và ước lượng tài nguyên là mục tiêu và ước lượng, không phải kết quả đo
- Cả 8 tài liệu LaTeX đã được biên dịch bằng XeLaTeX (`build_all.sh`): không có lỗi, không có tham chiếu chưa định nghĩa; chỉ còn cảnh báo overfull/underfull nhỏ trong bảng. Ngoài ra đã kiểm nhất quán số liệu và tên tín hiệu giữa các tài liệu

- Hai sơ đồ SVG đã qua lint của skill `bw-diagram` (kết quả OK) và một lượt tự kiểm tọa độ (dây nối đúng chân, không có chữ đè dây, không có khối chồng nhau)
- Các mô hình Python (`gemm_*_model.py`) nằm ở `gemm-accel/model/`. Ngày 12/11/2026 đã sửa `gemm_regfile_model.py` và `gemm_sched_model.py` theo D-01/D-03 (bỏ chế độ đơn, CTRL.START = bit 2, `HW_CONFIG` = 0x1100_2008) và chạy lại toàn bộ bằng `makemodels`: tất cả đạt. Các phép kiểm số học đã chạy riêng khi soạn baseline: requantize 34/33 bit khớp từng bit trên 800 576 trường hợp, công thức COMPUTE khớp lịch với 7 kích thước, dung lượng bank và số beat
- Review kiến trúc độc lập (đọc tài liệu, không mô phỏng) tìm 8 điểm: 2 blocker (D-34), 3 major (rcnt trong mục chuỗi descriptor của tài liệu hệ thống, D-35), 1 minor (liệt kê đường tổ hợp input → output), 2 nit (ký hiệu truy cập bit dự trữ CTRL, đánh số D-xx). Tất cả đã được sửa trong bộ tài liệu này